AJAY.I - 71812401007 · AGENTIC AI COGNITION · MODULE 3 — AGENT EVALUATION AND EMERGING TRENDS NOTES

1. METACOGNITION IN AI AGENTS

1.1 What is Metacognition?

Simple explanation:

Metacognition is nothing but the ability of an AI agent to think about its own thinking.

Normally, an AI agent solves a problem and provides an answer.

But a metacognitive AI agent does something more. It examines its own reasoning and asks:

- Am I solving this problem correctly?
- Is my answer reliable?
- Do I have enough information?
- Should I try another approach?
- Should I ask a human for help?

For example, imagine you are solving a difficult mathematical problem.

You calculate an answer, check your calculations and realize that you made a mistake. You then change your approach.

This is metacognition.

Technical explanation:

Metacognition in AI refers to the ability of an intelligent system to monitor, evaluate and regulate its own reasoning processes, decisions and problem-solving activities.

It enables an AI agent to assess its performance, recognize limitations, identify uncertainty and modify its reasoning strategy when necessary.

1.2 Why is Metacognition needed?

An ordinary AI agent may produce an incorrect answer without recognizing that it is incorrect.

Metacognition helps the agent evaluate its own performance before taking further action.

For example:

An AI coding agent generates a Python program.

1. It checks the generated code.
2. It identifies a possible error.
3. It decides to execute the program.
4. It examines the output.
5. It corrects the code if required.

The agent is not just solving the problem; it is also monitoring how well it is solving the problem.

1.3 Main Components of Metacognition

1. Metacognitive Knowledge

Simple explanation: The agent understands its own capabilities and limitations.

Example: An AI agent knows that it can perform mathematical calculations but may need an external tool for complex calculations.

Technical explanation: Metacognitive knowledge refers to information about the agent's reasoning abilities, available resources, limitations and task requirements.

2. Metacognitive Monitoring

Simple explanation: The agent continuously checks whether its current approach is working.

Example: A coding agent checks whether its program is producing the expected output.

Technical explanation: Metacognitive monitoring evaluates the progress, correctness and reliability of ongoing reasoning or actions.

3. Metacognitive Control

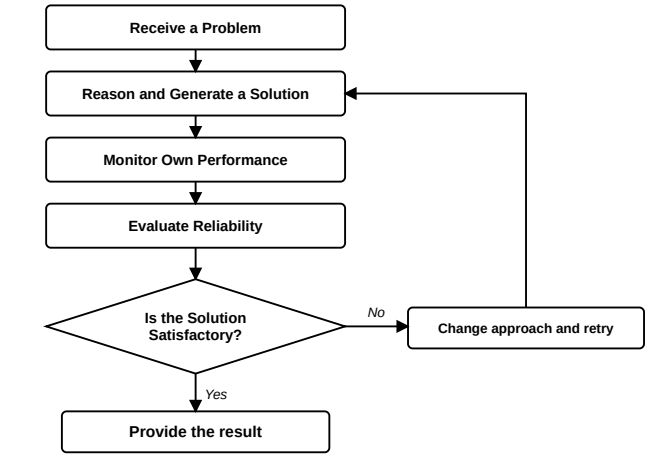
Simple explanation: The agent changes its approach when it identifies a problem.

Example: If one algorithm is too slow, the agent selects a more efficient algorithm.

Technical explanation: Metacognitive control involves modifying reasoning strategies, allocating additional resources or selecting alternative actions based on monitoring results.

1.4 Working of Metacognition

DIAGRAM



Working of metacognition

1.5 Example

Suppose an AI agent is asked to calculate the average of 10, 20, 30 and 40.

- Initial calculation: 30.
- Monitoring: Check the calculation.
- Evaluation: Total = 100, number of values = 4.
- Verification: 100 / 4 = 25.
- Correction: Replace the incorrect answer with 25.

The agent monitors and corrects its own reasoning.

1.6 Applications

- Autonomous coding agents.
- Medical decision-support systems.
- Robotics.
- Mathematical problem-solving.
- Self-correcting AI systems.
- Autonomous research agents.

1.7 Advantages

- Helps identify reasoning mistakes.
- Improves decision-making.
- Supports self-correction.
- Helps recognize limitations.
- Enables adaptive problem-solving.

Important: Metacognition means thinking about and evaluating one's own thinking. It is not simply generating an answer.

2. UNCERTAINTY QUANTIFICATION

2.1 What is Uncertainty Quantification?

Simple explanation:

Uncertainty Quantification is nothing but measuring how confident an AI agent is about its answer.

Imagine asking an AI:

"Will it rain tomorrow?"

The AI may predict:

- 90% chance of rain.
- 50% chance of rain.
- 10% chance of rain.

These values indicate different levels of uncertainty.

An AI agent should not always behave as if its answers are 100% correct.

It should identify when it does not have enough information.

Technical explanation:

Uncertainty Quantification (UQ) is the process of identifying, estimating and representing uncertainty in AI predictions, decisions and outputs.

It helps determine how reliable a model's predictions are and whether additional information, verification or human intervention is required.

2.2 Why is Uncertainty Quantification needed?

Suppose an AI agent is used in a healthcare application.

It predicts that a patient may have a particular disease.

If the model has high uncertainty, it should not confidently provide a final diagnosis.

Instead, the system may request additional medical tests or refer the case to a healthcare professional.

Therefore, uncertainty quantification is important in situations where incorrect decisions can cause serious consequences.

2.3 Types of Uncertainty

There are two major types.

1. Aleatoric Uncertainty

Simple explanation:

Aleatoric uncertainty means uncertainty caused by randomness or natural variation in the data.

Example:

A weather prediction may be uncertain because weather conditions naturally change.

Even if the model has perfect knowledge, some randomness cannot be eliminated.

Technical explanation:

Aleatoric uncertainty represents inherent randomness in observations or the underlying data-generating process.

It is generally not eliminated simply by collecting more training data.

2. Epistemic Uncertainty

Simple explanation:

Epistemic uncertainty means uncertainty caused by a lack of knowledge.

Example:

An AI model trained using images of cats and dogs receives an image of an unusual animal it has never encountered.

It may be uncertain because it lacks sufficient training knowledge.

Technical explanation:

Epistemic uncertainty arises from limited knowledge about the model or the underlying problem.

It can often be reduced by collecting additional representative data or improving the model.

2.4 Comparison

Aleatoric Uncertainty	Epistemic Uncertainty
Caused by natural randomness.	Caused by lack of knowledge.
Related to data variability.	Related to model limitations.
Generally cannot be removed by more training data alone.	Can often be reduced with additional data.
Example: Random sensor noise.	**Example:** Unseen data patterns.

2.5 Methods of Uncertainty Estimation

1. Probability-Based Estimation

Simple explanation: The model provides probabilities for different possible outcomes.

Example:

```
Cat: 80%
Dog: 15%
Other: 5%
```

Technical explanation: Probabilistic prediction represents uncertainty using probability distributions over possible outcomes.

2. Bayesian Methods

Simple explanation: The model considers different possible parameter values instead of depending on only one fixed set of values.

Technical explanation: Bayesian methods represent uncertainty using probability distributions over model parameters or predictions.

3. Monte Carlo Dropout

Simple explanation: The model makes several predictions with dropout enabled and compares the results.

Technical explanation: Monte Carlo Dropout uses multiple stochastic forward passes through a neural network to approximate predictive uncertainty.

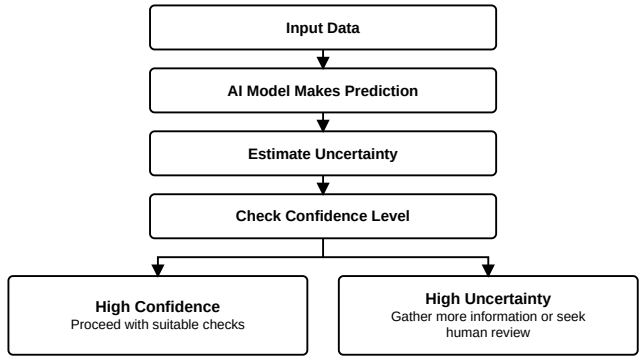
4. Ensemble Methods

Simple explanation: Multiple models make predictions, and their outputs are compared.

Technical explanation: Ensemble-based uncertainty estimation uses variation among multiple independently trained or differently configured models as an indication of predictive uncertainty.

2.6 Working of Uncertainty Quantification

DIAGRAM



Working of uncertainty quantification

2.7 Applications

- Weather forecasting.
- Medical diagnosis support.
- Autonomous vehicles.
- Financial risk prediction.
- Robotics.
- AI decision-making systems.

Important: High confidence does not automatically mean a prediction is correct. The model's confidence should be calibrated and evaluated against actual outcomes.

3. HUMAN-IN-THE-LOOP REASONING

3.1 What is Human-in-the-Loop?

Simple explanation:

Human-in-the-Loop (HITL) means involving a human in the AI decision-making process.

Imagine an AI system used to approve bank loans.

The AI analyzes the application and recommends whether the loan should be approved.

But instead of allowing the AI to make every final decision automatically, a human officer reviews important or uncertain cases.

This is Human-in-the-Loop.

Technical explanation:

Human-in-the-Loop reasoning is an AI architecture in which human feedback, supervision or approval is integrated into the reasoning and decision-making process of an AI system.

It combines automated AI capabilities with human judgment to improve reliability, safety and decision quality.

3.2 Why is HITL needed?

AI agents may sometimes:

- Generate incorrect information.
- Misunderstand user instructions.
- Make unsafe decisions.
- Lack sufficient context.
- Face situations requiring human judgment.

Human involvement helps identify and correct such problems.

3.3 Components of HITL

1. AI Agent

Simple explanation: The AI performs the task and generates a recommendation.

2. Human Reviewer

Simple explanation: A person checks the AI's recommendation.

3. Feedback Mechanism

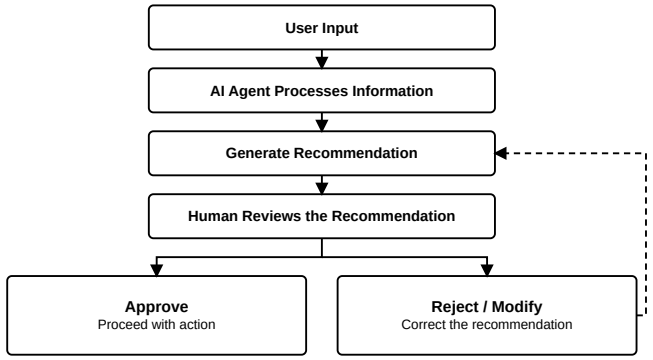
Simple explanation: The human provides corrections or suggestions.

4. Decision System

Simple explanation: The system uses the human feedback to approve, reject or modify the action.

3.4 Working of HITL

DIAGRAM



Working of Human-in-the-Loop

3.5 Types of Human-in-the-Loop Systems

1. Human-in-the-Loop (HITL)

Simple explanation: Humans actively participate in decisions or approve important actions.

Example: A doctor reviews an AI-generated diagnosis before treatment.

2. Human-on-the-Loop (HOTL)

Simple explanation: The AI works automatically, but a human supervises the process and can intervene when necessary.

Example: An operator monitors an autonomous manufacturing robot.

3. Human-out-of-the-Loop (HOOTL)

Simple explanation: The AI operates independently without continuous human intervention.

Example: An automated system performs routine data processing without requiring a person to approve every operation.

The level of human supervision depends on the application's risk and requirements.

3.6 Applications

- Healthcare.
- Banking and financial services.
- Autonomous vehicles.
- Cybersecurity.
- Industrial automation.
- Legal document analysis.

3.7 Advantages

- Improves decision reliability.
- Provides human judgment.
- Helps identify AI errors.
- Supports safer autonomous systems.
- Improves accountability.

3.8 Limitations

- Requires human time and resources.
- Can slow down automated workflows.
- Human reviewers may also make mistakes.
- Requires a clear approval and escalation process.

Important: HITL does not mean humans must perform every task. It means human judgment is incorporated where necessary.

4. AGENT EVALUATION METRICS

4.1 What is Agent Evaluation?

Simple explanation:

Agent Evaluation is nothing but checking how well an AI agent performs its assigned tasks.

Imagine you develop an AI coding agent.

How will you measure its performance?

You can check:

- How many coding problems it solves.
- How many attempts it requires.
- How much time it takes.
- How many errors it produces.
- How much it costs to complete a task.

These measurements are called evaluation metrics.

Technical explanation:

Agent Evaluation Metrics are quantitative or qualitative measures used to assess the performance, reliability, efficiency, accuracy and safety of autonomous AI agents.

They help determine whether an agent successfully achieves its objectives under specified conditions.

4.2 Important Agent Evaluation Metrics

1. Task Success Rate

Simple explanation:

It measures how many tasks the AI agent successfully completes.

Technical explanation:

Task Success Rate is the percentage of attempted tasks that are completed successfully.

Success Rate = (Successful Tasks / Total Tasks) × 100

Example:

An AI agent successfully completes 80 out of 100 tasks.

= (80 / 100) × 100 = 80%

2. Accuracy

Simple explanation:

Accuracy measures how many predictions are correct.

Technical explanation:

Accuracy is the proportion of correct predictions among all predictions made by a model.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Where:

- TP = True Positive.
- TN = True Negative.
- FP = False Positive.
- FN = False Negative.

3. Precision

Simple explanation:

Precision measures how many of the items predicted as positive are actually positive.

Precision = TP / (TP + FP)

4. Recall

Simple explanation:

Recall measures how many actual positive items the model successfully identifies.

Recall = TP / (TP + FN)

5. F1-Score

Simple explanation:

F1-score combines precision and recall into one measurement.

F1 = 2 × (Precision × Recall) / (Precision + Recall)

6. Response Time / Latency

Simple explanation:

It measures how long the AI agent takes to produce a response or complete an operation.

Example: An agent completes a task in 5 seconds.

7. Cost per Task

Simple explanation:

It measures how much money or computational resources are required to complete one task.

Example: An AI agent spends ₹2 worth of API usage to complete one task.

8. Tool-Call Success Rate

Simple explanation:

It measures how often the agent successfully uses its external tools.

Example: An agent successfully executes 95 out of 100 tool calls.

9. Robustness

Simple explanation:

It measures how well an AI agent performs when it faces unexpected inputs or changing situations.

Example: A chatbot should still work when users ask questions using different wording.

10. Safety

Simple explanation:

It measures whether the AI agent avoids harmful, unsafe or unauthorized actions.

Example: A financial agent should not transfer money without the required authorization.

4.3 Comparison of Metrics

Metric	What it measures
Success Rate	Completed tasks
Accuracy	Correct predictions
Precision	Reliability of positive predictions
Recall	Ability to identify actual positive cases
F1-Score	Balance between precision and recall
Latency	Time required
Cost per Task	Financial or computational cost
Tool-Call Success	Successful tool usage
Robustness	Performance under changing conditions
Safety	Avoidance of harmful actions

4.4 Why are Evaluation Metrics important?

- Compare different AI agents.
- Identify performance weaknesses.
- Measure improvements.
- Evaluate reliability.
- Monitor computational costs.
- Ensure safety and quality.

Important: Accuracy alone is not enough to evaluate an autonomous agent. A good agent must also be efficient, reliable, safe and capable of completing its actual tasks.

5. SWE-BENCH

5.1 What is SWE-bench?

Simple explanation:

SWE-bench is nothing but a test platform used to check whether an AI agent can solve real-world software programming problems.

Imagine a software company has a GitHub project.

Someone reports a bug:

"The login page crashes when the user enters invalid credentials."

Can an AI coding agent understand the problem, find the faulty code, modify it and fix the issue?

SWE-bench evaluates this type of ability.

Technical explanation:

SWE-bench is a benchmark designed to evaluate the ability of language models and AI coding agents to resolve real-world software engineering issues.

It uses issues collected from real GitHub repositories along with associated codebases and tests.

The agent must understand the issue, inspect the repository, modify the source code and produce a patch that satisfies the required tests.

5.2 Why is SWE-bench needed?

A normal coding test may ask:

"Write a Python program to sort an array."

But real software engineering involves much more than writing small programs.

A coding agent must:

- Understand an existing codebase.
- Identify the source of a bug.
- Understand the issue description.
- Modify existing files.
- Avoid breaking other functionality.
- Pass the required tests.

SWE-bench evaluates these practical capabilities.

5.3 Main Components

1. GitHub Repository

Simple explanation: Contains the actual software project that needs to be modified.

2. Issue Description

Simple explanation: Explains the bug or problem that needs to be solved.

3. AI Coding Agent

Simple explanation: Examines the code and attempts to fix the issue.

4. Patch Generation

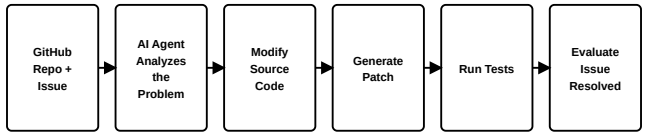
Simple explanation: The agent produces changes to the existing source code.

5. Test Cases

Simple explanation: Tests whether the changes solve the problem without causing failures.

5.4 Working of SWE-bench

DIAGRAM



Working of SWE-bench

5.5 Versions of SWE-bench

1. SWE-bench Full

Simple explanation: Tests coding agents using the full collection of software engineering issues in the benchmark.

2. SWE-bench Lite

Simple explanation: Uses a smaller selected subset of problems to make evaluation more manageable.

3. SWE-bench Verified

Simple explanation: Uses a subset of problems that have been manually checked for issue clarity and test quality.

It contains 500 human-validated tasks from SWE-bench.

5.6 Evaluation Metric

The major metric is the percentage of software issues successfully resolved.

Resolution Rate = (Issues Resolved Successfully / Total Issues) × 100

Example:

If an AI agent successfully resolves 60 out of 100 issues:

= (60 / 100) × 100 = 60%

5.7 Advantages

- Evaluates practical coding abilities.
- Uses real-world software problems.
- Tests code modification and debugging.
- Measures issue-resolution performance.
- Helps compare coding agents.

5.8 Limitations

- Requires significant computational resources.
- Results depend on the test environment.
- Passing tests does not guarantee perfect software quality.
- Different agents may use different tools and execution strategies.

Important: SWE-bench primarily evaluates software engineering and real-world GitHub issue resolution.

6. AGENTBENCH

6.1 What is AgentBench?

Simple explanation:

AgentBench is nothing but a testing platform used to evaluate how well AI agents perform different types of tasks in interactive environments.

Imagine testing a student in different subjects:

- Mathematics.
- Science.
- Computer Science.
- English.

Testing only mathematics cannot tell you the student's complete ability.

Similarly, testing an AI agent only on coding problems cannot tell you its overall intelligence.

AgentBench evaluates AI agents across different environments and tasks.

Technical explanation:

AgentBench is a benchmark framework designed to evaluate the capabilities of large language model-based agents across multiple interactive environments.

It assesses how effectively agents perceive their environments, reason, make decisions, use tools and complete tasks.

6.2 Why is AgentBench needed?

AI agents are designed to perform different types of tasks.

For example:

- Interacting with databases.
- Operating websites.
- Solving games.
- Writing and executing code.
- Using operating-system commands.

AgentBench provides a structured way to evaluate these capabilities.

6.3 Main Evaluation Environments

AgentBench's original benchmark includes eight interactive environments.

Environment	What it tests
Operating System	Ability to perform OS-related tasks
Database	Ability to retrieve and manipulate database information
Knowledge Graph	Ability to answer questions using graph information
Digital Card Game	Strategic decision-making
Lateral Thinking Puzzles	Creative and logical reasoning
House-Holding	Completing tasks in an interactive household environment
Web Shopping	Finding and purchasing products in a simulated environment
Web Browsing	Navigating websites and retrieving information

6.4 Working of AgentBench

AI Agent → Select Evaluation Task → Interact with Environment → Execute Actions and Make Decisions → Evaluate Task Completion → Generate Performance Results

6.5 Example

Suppose an AI agent is tested in a web-browsing environment.

Task:

"Find a product with a price below ₹1,000."

The agent must:

1. Open the website.
2. Search for the product.
3. Examine the available results.
4. Compare prices.
5. Select a suitable product.
6. Report the result.

The benchmark evaluates whether the agent successfully completes the task.

6.6 Advantages

- Evaluates different agent capabilities.
- Tests interaction with environments.
- Supports comparison between models.
- Identifies strengths and weaknesses.
- Measures autonomous task completion.

6.7 Limitations

- Benchmark environments may not represent every real-world situation.
- Results depend on the selected tasks.
- Some environments may require substantial resources.
- High benchmark performance does not guarantee perfect real-world performance.

7. BENCHMARKING AUTONOMOUS AGENTS

7.1 What is Benchmarking?

Simple explanation:

Benchmarking is nothing but comparing the performance of different AI agents using the same set of tests.

Imagine two AI coding agents.

You give both agents the same 100 programming problems.

- Agent A solves 85 problems.
- Agent B solves 70 problems.

You can compare their performance using the same evaluation conditions.

This is benchmarking.

Technical explanation:

Benchmarking Autonomous Agents is the systematic evaluation and comparison of autonomous AI systems using standardized tasks, environments, datasets and performance metrics.

It measures capabilities such as task completion, reasoning, tool usage, efficiency, robustness and safety.

7.2 Why is Benchmarking important?

Benchmarking helps developers understand:

- Which agent performs better.
- Which agent completes tasks faster.
- Which agent uses fewer resources.
- Which agent makes fewer mistakes.
- Which agent is more reliable.
- Which agent performs better in unfamiliar situations.

7.3 Steps involved in Benchmarking

Step 1: Define the Objective

Decide what capability you want to evaluate.

Example: Coding, reasoning or web browsing.

Step 2: Select a Benchmark

Choose a suitable benchmark.

Examples:

- SWE-bench for software engineering.
- AgentBench for multiple interactive environments.

Step 3: Prepare the Environment

Provide the same tasks, resources and execution conditions to the agents.

Step 4: Execute the Agents

Run each agent against the selected benchmark tasks.

Step 5: Collect Results

Record the number of successful tasks, execution time, costs and errors.

Step 6: Compare Performance

Compare the collected metrics to identify the best-performing agent for the intended use case.

7.4 Important Benchmarking Metrics

Metric	Purpose
Success Rate	Measures successful task completion
Accuracy	Measures correctness
Latency	Measures execution time
Cost	Measures resource expenditure
Robustness	Measures performance under changing conditions
Reliability	Measures consistency of results
Safety	Measures harmful or unauthorized actions
Tool Efficiency	Measures effective use of tools

7.5 Example

Suppose two agents are tested on 100 tasks.

Metric	Agent A	Agent B
Successful tasks	85	75
Success rate	85%	75%
Average time	8 seconds	5 seconds
Cost per task	₹2	₹1

Agent A has a higher success rate, but Agent B is faster and cheaper.

Therefore, the best agent depends on the application's requirements.

For a critical task, accuracy may be more important. For a large-scale simple task, speed and cost may matter more.

7.6 Challenges in Benchmarking

- Different agents may use different tools.
- Some tasks may be easier than others.
- Evaluation conditions may affect results.
- Benchmarks may not cover every real-world scenario.
- Agents may perform differently on unseen tasks.

Important: SWE-bench and AgentBench are specific benchmarks, whereas benchmarking is the overall process of systematically evaluating and comparing

agents.

8. SELF-IMPROVING AGENTS

8.1 What is a Self-Improving Agent?

Simple explanation:

A Self-Improving Agent is nothing but an AI agent that uses its previous performance and feedback to improve how it performs future tasks.

Imagine an AI coding agent.

Initially, it writes code with several errors.

After repeated attempts, it:

1. Executes the code.
2. Identifies errors.
3. Analyzes the failures.
4. Improves its approach.
5. Stores useful experiences.
6. Performs better on similar tasks.

This is the basic idea of a self-improving agent.

Technical explanation:

A Self-Improving Agent is an autonomous AI system that uses feedback, evaluation, memory, experience or learning mechanisms to improve its future performance.

It may improve its strategies, prompts, tool selection, memory, planning or model parameters depending on its architecture.

8.2 Why is Self-Improvement needed?

Traditional AI systems often depend on fixed instructions and predefined behavior.

However, autonomous agents face changing tasks and environments.

Self-improvement allows agents to adapt based on their previous experiences.

8.3 Main Components

1. Performance Monitoring

Simple explanation: Checks how well the agent performs its tasks.

2. Feedback Collection

Simple explanation: Collects information about mistakes and successful actions.

3. Memory System

Simple explanation: Stores previous experiences and useful knowledge.

4. Improvement Mechanism

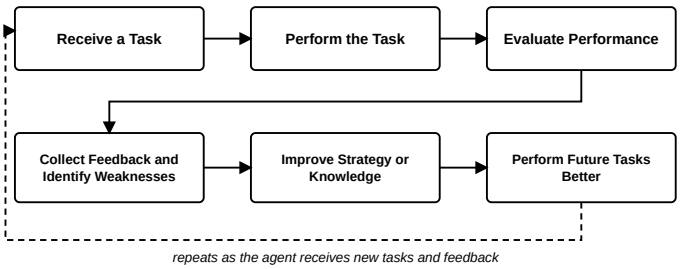
Simple explanation: Changes the agent's strategy to improve future performance.

5. Evaluation

Simple explanation: Checks whether the improvements actually work.

8.4 Working of Self-Improving Agents

DIAGRAM



Working of self-improving agents

The improvement process repeats as the agent receives new tasks and feedback.

8.5 Methods of Self-Improvement

1. Feedback-Based Improvement

Simple explanation: The agent improves using feedback from humans or evaluation systems.

Example: A human reviewer identifies mistakes in a generated report.

2. Experience-Based Improvement

Simple explanation: The agent uses previously stored experiences to solve new problems.

Example: A coding agent reuses a successful debugging strategy.

3. Reinforcement Learning

Simple explanation: The agent learns by receiving rewards for good actions and penalties for poor actions.

Technical explanation: Reinforcement learning improves an agent's policy through interactions with an environment and feedback in the form of rewards.

4. Self-Reflection

Simple explanation: The agent examines its previous work and identifies ways to improve.

Example: An agent reviews why a particular task failed.

5. Model Updating

Simple explanation: The underlying model itself is trained or fine-tuned to improve performance.

Technical explanation: Model updating modifies model parameters using training or fine-tuning procedures based on suitable data and optimization objectives.

8.6 Advantages

- Supports continuous improvement.
- Helps learn from previous mistakes.
- Adapts to changing environments.
- Improves task performance.
- Reduces repeated failures.

8.7 Limitations

- Incorrect feedback can reduce performance.
- Uncontrolled updates may introduce new errors.
- Requires evaluation and monitoring.
- Can increase computational costs.
- Self-improvement does not guarantee unlimited intelligence or correctness.

9. DISTILLATION OF SYSTEM 2 INTO SYSTEM 1

This is one of the most important and interesting topics in Module 3.

9.1 What are System 1 and System 2?

Before understanding distillation, you need to understand these two reasoning systems.

System 1 Reasoning

Simple explanation:

System 1 is nothing but fast and automatic thinking.

For example:

If someone asks:

"What is 2 + 2?"

You immediately answer 4 without spending much time thinking.

This is System 1.

Technical explanation:

System 1 refers to fast, automatic and computationally efficient reasoning that produces responses with relatively little deliberate computation.

System 2 Reasoning

Simple explanation:

System 2 is nothing but slow, careful and detailed thinking.

For example:

If someone asks you to solve a complex mathematical problem, you take time to analyze the question, calculate the answer and verify your result.

This is System 2.

Technical explanation:

System 2 refers to deliberate, multi-step reasoning that uses additional computation to solve complex problems, evaluate alternatives and verify solutions.

9.2 Comparison of System 1 and System 2

System 1	System 2
Fast reasoning.	Deliberate reasoning.
Usually automatic.	Requires additional reasoning.
Low computational cost.	Generally higher computational cost.
Suitable for routine tasks.	Suitable for complex tasks.
Example: Simple arithmetic.	**Example:** Complex mathematical problem.

9.3 What is Distillation?

Simple explanation:

Distillation is nothing but teaching a smaller or faster AI model to reproduce useful capabilities of a more powerful model.

Imagine a teacher solving a difficult problem using 20 steps.

After understanding the solution, a student learns a simpler method that solves similar problems using only 5 steps.

The student can now solve the problem faster without repeating the teacher's complete process.

This is the basic idea of distillation.

Technical explanation:

Knowledge Distillation is a model compression and knowledge-transfer technique in which a student model learns to approximate the behavior or outputs of a larger teacher model.

It uses teacher-generated predictions, explanations, labels or other training signals to train the student model.

9.4 What is Distillation of System 2 into System 1?

Simple explanation:

It means taking the useful problem-solving knowledge developed through slow and careful reasoning and transferring it into a faster reasoning system.

The objective is to make future tasks faster and more efficient without requiring the agent to perform the entire expensive reasoning process every time.

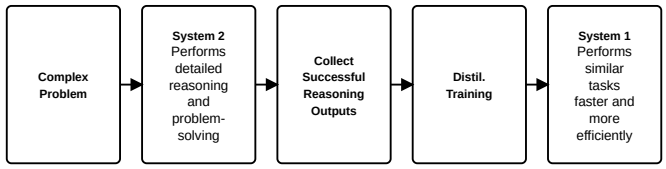
Technical explanation:

Distillation of System 2 into System 1 refers to transferring capabilities developed through computationally intensive reasoning into a more efficient model or policy that can perform similar tasks with lower inference cost.

The distilled system learns patterns, strategies or solutions from the more deliberate reasoning process.

9.5 Working of Distillation

DIAGRAM



Distillation of System 2 into System 1

9.6 Example

Suppose an AI agent must solve 1,000 mathematical problems.

Initially:

- A powerful reasoning model solves each problem using detailed reasoning.
- It generates correct solutions.
- These solutions are collected as training data.
- A smaller model is trained using these examples.

After training, the smaller model can solve many similar problems with less computation.

9.7 Advantages

- Reduces inference cost.
- Improves response speed.
- Transfers useful reasoning capabilities.
- Reduces computational requirements.
- Makes complex capabilities more efficient to deploy.

9.8 Limitations

- The student may lose some advanced reasoning capabilities.
- Incorrect teacher outputs can affect training.
- Requires suitable training data.
- The distilled model may fail on unfamiliar problems.

Important: Distillation does not mean that System 1 becomes identical to System 2. It means transferring selected capabilities into a more efficient system.

10. FUTURE TRENDS IN AGENTIC AI

10.1 What are Future Trends?

Simple explanation:

Future Trends in Agentic AI means understanding how autonomous AI systems may develop and what new capabilities they may support in the coming years.

Today, many AI systems respond to user questions.

Future agentic systems are increasingly being designed to perform complete tasks, interact with software, use multiple tools and collaborate with other agents.

Technical explanation:

Future trends in Agentic AI involve emerging research directions, architectures and technologies that improve autonomous reasoning, planning, learning, collaboration, multimodal perception and real-world task execution.

10.2 Important Future Trends

1. Multi-Agent Systems

Simple explanation:

Multiple AI agents work together to complete a large task.

Example:

For developing a website:

- Agent 1: Frontend development.
- Agent 2: Backend development.
- Agent 3: Database management.
- Agent 4: Testing.

All agents coordinate to complete the project.

Technical explanation:

Multi-Agent Systems consist of multiple autonomous agents that interact, coordinate and collaborate to solve complex tasks.

2. Multimodal AI Agents

Simple explanation:

AI agents will understand different types of information, such as text, images, audio and video.

Example:

A robot can understand a spoken instruction, recognize an object through a camera and perform a physical action.

Technical explanation:

Multimodal AI agents integrate different data modalities to perceive, reason and act in complex environments.

3. Self-Improving Agents

Simple explanation:

AI agents will become better at performing tasks by learning from their previous experiences and feedback.

Example: A coding agent improves its debugging strategies over time.

4. Autonomous Robotics

Simple explanation:

AI agents will control robots that can perform tasks independently.

Example: Warehouse robots that identify objects and transport packages.

5. AI Personal Assistants

Simple explanation:

AI assistants will perform complete tasks rather than only answering questions.

Example: An assistant that organizes meetings, prepares documents and manages permitted calendar operations.

6. Long-Term Memory Systems

Simple explanation:

AI agents will remember previous tasks, important information and user preferences across different sessions.

Example: An AI assistant that continues a research project using previously stored information.

7. Agentic Software Engineering

Simple explanation:

AI agents will increasingly participate in the complete software development process.

Example:

- Understand requirements.
- Generate code.
- Run tests.
- Identify bugs.
- Fix errors.
- Prepare deployment changes.

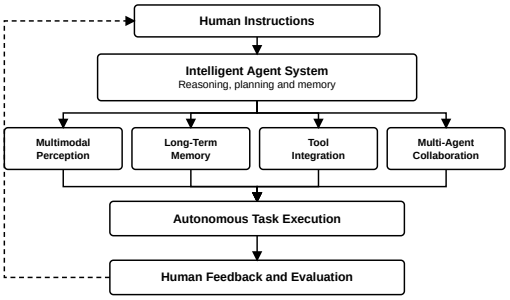
8. Improved Human–AI Collaboration

Simple explanation:

Humans and AI agents will work together, with AI handling repetitive tasks and humans supervising important decisions.

10.3 Future Agentic AI Architecture

DIAGRAM



Future agentic AI architecture

10.4 Advantages of Future Agentic AI

- Automates complex workflows.
- Improves productivity.
- Supports personalized services.
- Enables intelligent decision-making.
- Improves human–AI collaboration.

11. ETHICAL AND RESPONSIBLE AI AGENTS

11.1 What is Responsible AI?

Simple explanation:

Responsible AI means developing AI systems that are safe, fair, transparent and respectful of human rights.

Imagine an AI system used by a bank to approve loans.

If the system rejects applicants unfairly based on irrelevant personal characteristics, it creates an ethical problem.

Therefore, AI systems must be designed and used responsibly.

Technical explanation:

Responsible AI refers to the principles, practices and governance mechanisms used to develop and deploy AI systems in ways that promote fairness, transparency, accountability, privacy, safety and human well-being.

11.2 Why is Responsible AI needed?

AI agents may:

- Make incorrect decisions.
- Produce biased results.
- Reveal private information.
- Perform unauthorized actions.
- Generate harmful content.
- Operate without sufficient human supervision.

Responsible AI helps reduce these risks.

11.3 Principles of Responsible AI

1. Fairness

Simple explanation:

AI should treat people fairly and should not discriminate against individuals or groups.

Example:

A recruitment AI should evaluate candidates based on relevant qualifications rather than irrelevant personal characteristics.

Technical explanation:

Fairness aims to prevent unjustified bias and discriminatory outcomes in AI systems.

2. Transparency

Simple explanation:

Users should understand how an AI system works and why it produces certain decisions, to the extent possible.

Example:

A loan approval system should provide understandable reasons for its recommendations.

Technical explanation:

Transparency involves communicating relevant information about the AI system's capabilities, limitations, data usage and decision processes.

3. Accountability

Simple explanation:

Someone must be responsible for the decisions and actions of an AI system.

Example:

A company deploying an AI-based financial agent must establish responsibility for monitoring and controlling its actions.

Technical explanation:

Accountability establishes clear responsibility for AI development, deployment, monitoring and consequences.

4. Privacy

Simple explanation:

AI systems must protect users' personal and confidential information.

Example:

A medical AI system should prevent unauthorized access to patient records.

Technical explanation:

Privacy involves protecting personal data through suitable collection, processing, storage and access-control practices.

5. Safety and Security

Simple explanation:

AI systems should not perform harmful or unauthorized actions.

Example:

An autonomous agent should not delete important files without the required authorization.

Technical explanation:

AI safety and security involve preventing harmful behavior, unauthorized access, system manipulation and unintended actions.

6. Human Oversight

Simple explanation:

Humans should be able to supervise or intervene in important AI decisions.

Example:

A human doctor reviews an AI-generated medical recommendation.

Technical explanation:

Human oversight establishes mechanisms for human supervision, intervention and control over AI operations.

7. Reliability

Simple explanation:

AI systems should work consistently and produce dependable results.

Example:

A navigation agent should provide reliable directions under different operating conditions.

Technical explanation:

Reliability refers to the ability of an AI system to perform consistently under expected conditions and recover appropriately from failures.

11.4 Ethical Challenges in Agentic AI

1. Bias

AI agents may produce unfair outcomes because of biased training data or design.

2. Hallucination

AI agents may generate information that appears correct but is actually false.

3. Privacy Violations

Agents may expose sensitive information if data is not properly protected.

4. Unauthorized Actions

An autonomous agent may perform an operation without the necessary permission.

5. Lack of Accountability

It may be difficult to identify who is responsible when an autonomous agent causes harm.

6. Overdependence on AI

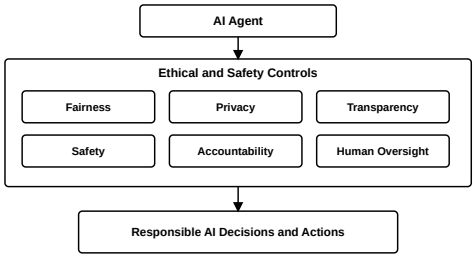
People may depend too heavily on AI decisions without checking their correctness.

11.5 Methods to Build Responsible AI Agents

1. Use representative and high-quality training data.
2. Test systems for bias.
3. Apply data encryption and access controls.
4. Use human approval for high-risk operations.
5. Maintain logs of important actions.
6. Monitor the system continuously.
7. Establish clear safety rules.
8. Regularly evaluate performance and reliability.

11.6 Responsible AI Architecture

DIAGRAM



Responsible AI architecture

11.7 Advantages of Responsible AI

- Reduces harmful AI decisions.
- Protects user privacy.
- Promotes fairness.
- Improves public trust.
- Supports safe deployment.
- Establishes accountability.

12. IMPORTANT COMPARISONS

These comparisons are useful for examination questions.

12.1 SWE-bench vs AgentBench

SWE-bench	AgentBench
Focuses on software engineering tasks.	Focuses on multiple interactive tasks.
Uses real GitHub issues.	Uses different interactive environments.
Tests code modification and bug fixing.	Tests reasoning, tool use and task completion.
Mainly evaluates coding agents.	Evaluates broader language-model agents.
Example: Fixing a software bug.	**Example:** Operating a website or database.

12.2 System 1 vs System 2

System 1	System 2
Fast reasoning.	Deliberate reasoning.
Automatic processing.	Detailed analysis.
Lower computational cost.	Higher computational cost.
Suitable for routine tasks.	Suitable for complex tasks.
Example: Simple arithmetic.	**Example:** Complex problem-solving.

12.3 Metacognition vs Self-Correction

Metacognition	Self-Correction
Thinking about one's own thinking.	Correcting identified mistakes.
Monitors and evaluates reasoning.	Revises an output or action.
Helps decide whether a change is required.	Performs the improvement.
Example: Recognizing uncertainty.	**Example:** Fixing incorrect code.

12.4 HITL vs HOTL vs HOOTL

Type	Human involvement
HITL	Human actively participates in decisions.
HOTL	Human supervises and can intervene.
HOOTL	AI operates without continuous human involvement.

13. IMPORTANT 2-MARK QUESTIONS

Prepare these definitions for short-answer questions.

1. Define Metacognition in AI.

Metacognition is the ability of an AI agent to monitor, evaluate and regulate its own reasoning and problem-solving processes.

2. What are the three components of Metacognition?

- Metacognitive Knowledge.
- Metacognitive Monitoring.
- Metacognitive Control.

3. Define Uncertainty Quantification.

Uncertainty Quantification is the process of measuring and representing uncertainty in AI predictions and decisions.

4. What is Aleatoric Uncertainty?

Aleatoric uncertainty is uncertainty caused by inherent randomness or variability in the data.

5. What is Epistemic Uncertainty?

Epistemic uncertainty is uncertainty caused by limited knowledge about the model or problem.

6. Expand HITL.

HITL stands for Human-in-the-Loop.

7. Define Human-in-the-Loop Reasoning.

Human-in-the-Loop reasoning integrates human feedback, supervision or approval into AI decision-making.

8. What is Human-on-the-Loop?

Human-on-the-Loop is an approach in which AI operates automatically under human supervision, allowing intervention when required.

9. What is Task Success Rate?

Task Success Rate is the percentage of tasks successfully completed by an AI agent out of the total attempted tasks.

10. Define Precision.

Precision is the proportion of correctly predicted positive cases among all predicted positive cases.

11. Define Recall.

Recall is the proportion of correctly identified positive cases among all actual positive cases.

12. What is SWE-bench?

SWE-bench is a benchmark used to evaluate AI agents on real-world software engineering issues collected from GitHub repositories.

13. What is SWE-bench Verified?

SWE-bench Verified is a subset of SWE-bench containing 500 human-validated software engineering tasks.

14. Define AgentBench.

AgentBench is a benchmark designed to evaluate language-model agents across multiple interactive environments and tasks.

15. What is Benchmarking?

Benchmarking is the systematic evaluation and comparison of AI agents using standardized tasks, environments and performance metrics.

16. Define Self-Improving Agents.

Self-Improving Agents are AI systems that use feedback, experience and learning mechanisms to improve their future performance.

17. What is System 1 Reasoning?

System 1 is fast, automatic and computationally efficient reasoning.

18. What is System 2 Reasoning?

System 2 is deliberate and computationally intensive reasoning used for complex problem-solving.

19. Define Knowledge Distillation.

Knowledge Distillation is a technique in which a student model learns to reproduce useful capabilities of a larger teacher model.

20. What is Distillation of System 2 into System 1?

It is the transfer of capabilities developed through detailed reasoning into a faster and more efficient reasoning system.

21. What is Responsible AI?

Responsible AI refers to developing and deploying AI systems with fairness, transparency, accountability, privacy and safety.

22. Mention any four principles of Responsible AI.

- Fairness.
- Transparency.
- Accountability.
- Privacy.